

Repository Structure & Architecture Guide

Welcome to the Wildfire CA Project! To ensure smooth collaboration across all three teams and avoid massive Git merge conflicts, this repository is strictly divided into distinct modules.

Please familiarize yourself with the structure below. ****You should primarily work within your team's designated directory.****

```
```text
wildfire-ca-project/
├── .gitignore # Ensures temporary files (e.g., __pycache__, .venv) stay out of Git
├── README.md # Main project description and setup instructions
├── requirements.txt # List of Python dependencies (e.g., numpy, matplotlib, pygame)
├── configs/ # Shared configuration files (JSON/YAML)
│ ├── default_scenario.json # Standard map size, fire origin, default parameters
│ └── wind_experiment.json # Config for specific analytical experiments
├── src/ # Main source code directory
│ ├── core/ # TEAM 1: Simulation Engine (Backend & Physics)
│ │ ├── __init__.py
│ │ ├── grid.py # 2D Grid representation and cell states
│ │ ├── rules.py # Propagation logic (Moore/von Neumann neighborhoods)
│ │ ├── engine.py # Main simulation loop (Must run strictly HEADLESS - no GUI)
│ │ └── interfaces.py # DTOs and API contracts for Teams 2 & 3
│ ├── gui/ # TEAM 2: Visualization & User Interface (Frontend)
│ │ ├── __init__.py
│ │ ├── app.py # Main UI window, buttons, and event loops
│ │ └── renderer.py # Translates the simulation state from core/ into visuals
│ └── analytics/ # TEAM 3: Strategy, AI, and Data Science
│ ├── __init__.py
│ ├── metrics.py # Data collection, logging, and plotting
│ ├── strategies.py # Baseline defense heuristics (e.g., random firebreaks)
│ └── ai_agent.py # Optional AI decision-making agent
├── tests/ # Unit and Integration Tests
│ ├── test_core.py # Tests for Team 1 (e.g., "Does fire spread correctly?")
│ └── test_api.py # Tests to ensure communication between modules works
```
```

Why is it structured this way? (The "API-First" Approach)

1. **Decoupling (Separation of Concerns):** The physics engine (`core`) should not know *how* it is being drawn. The visualizer (`gui`) should not know *how* the fire spreads. They only communicate through strictly defined functions and data objects found in `interfaces.py`.

2. **Preventing Git Conflicts:** By isolating your work into specific folders (`core/`, `gui/`, `analytics/`), Team 1 and Team 2 can work simultaneously without constantly overwriting each other's code.
3. **Headless Execution:** Team 3 needs to run thousands of simulations quickly to generate data plots or train an AI. If the engine is tangled with the GUI code, running fast background simulations becomes impossible. The `engine.py` must be fully independent of graphics.



Getting Started Workflow

1. **Agree on the API:** Before writing logic, all teams must sit down and agree on what the functions in `interfaces.py` will look like (e.g., `def get_grid_state() -> list:`, `def ignite(x: int, y: int):`).
2. **Branch Out:** Never push directly to `main`. Create a branch for your feature (e.g., `git checkout -b feature/wind-mechanics`).
3. **Pull Requests:** Once your feature is ready, open a Pull Request (PR). Have someone from *another* team review your code to ensure it doesn't break their module.